

Project 06: Blogging Platform

Difficulty: Intermediate | Estimated Time: 2 Weeks

1. Project Overview and Goals

This project builds the database backend for a multi-author blogging platform (think Medium or Dev.to). It covers user authentication, post management with versioning, rich taxonomies (tags, series, topics), commenting with threading, reactions, follows, notifications, search, and analytics. The blogging domain is content-heavy and read-intensive — making it an excellent project for learning full-text search and caching patterns.

Goals:

- Model user-generated content with revision history.
 - Implement threaded comments using an adjacency list and closure table.
 - Build a full-text search system using PostgreSQL's native FTS.
 - Design a notification fan-out system.
 - Write analytics queries for post performance (views, read time, engagement rate).
-

2. Learning Objectives

- Use `tsvector` columns with GIN indexes for fast full-text search.
 - Implement threaded comments with recursive CTEs.
 - Design a follows graph and write feed/timeline queries.
 - Build materialized views for expensive post metrics aggregation.
 - Practice array operations for tags.
 - Use `GENERATED ALWAYS AS` for computed columns.
 - Design an audit table for post revisions (content versioning).
 - Implement a polymorphic relationship pattern for notifications.
-

3. Functional Requirements

- **Users:** Registration, profiles, bio, social links, follower/following graph.
 - **Posts:** Title, content (Markdown), slug, cover image, status, tags, series.
 - **Revisions:** Full content history per post.
 - **Comments:** Threaded comments with moderation and reactions.
 - **Reactions:** Likes, claps, hearts — polymorphic (posts and comments).
 - **Tags/Topics:** Many-to-many tags with canonical slugs.
 - **Series:** Ordered collections of posts.
 - **Notifications:** Follow, comment, reaction, and mention notifications.
 - **Analytics:** Views, read time, engagement rate, trending posts.
-

4. Non-Functional Requirements

- Post slugs unique per author (not globally).
- Content stored as `TEXT` ; `search_vector` stored as `TSVECTOR` and updated via trigger.
- Comments support soft deletion with content replaced by "Deleted."
- Read counts stored in a separate high-write `post_views` table.

- All monetary support for tipping/monetization via `NUMERIC(10,2)` .

5. Complete Database Schema

```
-- =====
-- BLOGGING PLATFORM - Complete Schema
-- PostgreSQL 15+
-- =====

CREATE SCHEMA IF NOT EXISTS blog;
SET search_path = blog, public;

-- -----
-- USERS
-- -----

CREATE TABLE users (
    user_id      SERIAL PRIMARY KEY,
    username     VARCHAR(50) NOT NULL UNIQUE
                CHECK (username ~ '^[a-z0-9_-]{3,50}$'),
    email        VARCHAR(300) NOT NULL UNIQUE,
    display_name VARCHAR(200) NOT NULL,
    bio          TEXT,
    avatar_url   VARCHAR(500),
    website_url  VARCHAR(500),
    twitter_handle VARCHAR(50),
    github_handle VARCHAR(50),
    is_active    BOOLEAN NOT NULL DEFAULT TRUE,
    is_staff     BOOLEAN NOT NULL DEFAULT FALSE,
    email_verified BOOLEAN NOT NULL DEFAULT FALSE,
    post_count   INTEGER NOT NULL DEFAULT 0,
    follower_count INTEGER NOT NULL DEFAULT 0,
    following_count INTEGER NOT NULL DEFAULT 0,
    joined_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_seen_at TIMESTAMPTZ
);

CREATE INDEX idx_users_username ON users (username);
CREATE INDEX idx_users_email    ON users (email);
CREATE INDEX idx_users_active   ON users (is_active) WHERE is_active = TRUE;

-- -----
-- FOLLOWS (user graph)
-- -----

CREATE TABLE follows (
    follower_id INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    following_id INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    PRIMARY KEY (follower_id, following_id),
    CONSTRAINT chk_no_self_follow CHECK (follower_id <> following_id)
);
```

```

CREATE INDEX idx_follows_following ON follows (following_id);

-- -----
-- TOPICS (top-level categories)
-- -----

CREATE TABLE topics (
  topic_id SERIAL PRIMARY KEY,
  slug VARCHAR(100) NOT NULL UNIQUE,
  name VARCHAR(100) NOT NULL,
  description TEXT,
  color_hex VARCHAR(7) DEFAULT '#6C757D'
    CHECK (color_hex ~ '^#[0-9A-Fa-f]{6}$'),
  post_count INTEGER NOT NULL DEFAULT 0
);

-- -----
-- TAGS
-- -----

CREATE TABLE tags (
  tag_id SERIAL PRIMARY KEY,
  slug VARCHAR(100) NOT NULL UNIQUE,
  name VARCHAR(100) NOT NULL,
  topic_id INTEGER REFERENCES topics(topic_id),
  post_count INTEGER NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_tags_slug ON tags (slug);
CREATE INDEX idx_tags_topic ON tags (topic_id);

-- -----
-- SERIES
-- -----

CREATE TABLE series (
  series_id SERIAL PRIMARY KEY,
  author_id INTEGER NOT NULL REFERENCES users(user_id),
  slug VARCHAR(200) NOT NULL,
  title VARCHAR(400) NOT NULL,
  description TEXT,
  cover_url VARCHAR(500),
  is_complete BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (author_id, slug)
);

-- -----
-- POSTS
-- -----

CREATE TABLE posts (
  post_id SERIAL PRIMARY KEY,
  author_id INTEGER NOT NULL REFERENCES users(user_id),

```

```

slug          VARCHAR(300) NOT NULL,
title         VARCHAR(500) NOT NULL,
subtitle      VARCHAR(500),
content       TEXT NOT NULL,
content_format VARCHAR(10) NOT NULL DEFAULT 'markdown'
              CHECK (content_format IN ('markdown','html')),
cover_url     VARCHAR(500),
series_id     INTEGER REFERENCES series(series_id),
series_position SMALLINT,
status        VARCHAR(20) NOT NULL DEFAULT 'draft'
              CHECK (status IN ('draft','published','unlisted','archived')),
published_at  TIMESTAMPTZ,
estimated_read_min SMALLINT GENERATED ALWAYS AS
              (GREATEST(1, (CHAR_LENGTH(content) / 1300)::SMALLINT)) STORED,
view_count    INTEGER NOT NULL DEFAULT 0,
reaction_count INTEGER NOT NULL DEFAULT 0,
comment_count INTEGER NOT NULL DEFAULT 0,
search_vector TSVECTOR,
metadata      JSONB NOT NULL DEFAULT '{}',
created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
UNIQUE (author_id, slug)
);

```

```

CREATE INDEX idx_posts_author    ON posts (author_id, published_at DESC);
CREATE INDEX idx_posts_status   ON posts (status, published_at DESC);
CREATE INDEX idx_posts_series   ON posts (series_id, series_position);
CREATE INDEX idx_posts_fts      ON posts USING GIN (search_vector);
CREATE INDEX idx_posts_published ON posts (published_at DESC) WHERE status =
'published';
CREATE INDEX idx_posts_metadata ON posts USING GIN (metadata);

```

```

-- POST TAGS (many-to-many)

```

```

CREATE TABLE post_tags (
  post_id INTEGER NOT NULL REFERENCES posts(post_id) ON DELETE CASCADE,
  tag_id  INTEGER NOT NULL REFERENCES tags(tag_id),
  PRIMARY KEY (post_id, tag_id)
);

```

```

CREATE INDEX idx_post_tags_tag ON post_tags (tag_id);

```

```

-- POST REVISIONS

```

```

CREATE TABLE post_revisions (
  revision_id BIGSERIAL PRIMARY KEY,
  post_id     INTEGER NOT NULL REFERENCES posts(post_id) ON DELETE CASCADE,
  revision_num SMALLINT NOT NULL,
  title       VARCHAR(500),
  content     TEXT,

```

```

    revised_by    INTEGER NOT NULL REFERENCES users(user_id),
    revised_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    change_summary TEXT,
    UNIQUE (post_id, revision_num)
);

CREATE INDEX idx_revisions_post ON post_revisions (post_id, revision_num DESC);

-- -----
-- POST VIEWS (high-write analytics table)
-- -----

CREATE TABLE post_views (
    view_id       BIGSERIAL PRIMARY KEY,
    post_id       INTEGER NOT NULL REFERENCES posts(post_id) ON DELETE CASCADE,
    viewer_id     INTEGER REFERENCES users(user_id),
    session_hash  VARCHAR(64), -- hashed session for anonymous tracking
    read_pct      SMALLINT CHECK (read_pct BETWEEN 0 AND 100),
    viewed_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
) PARTITION BY RANGE (viewed_at);

-- Monthly partitions
CREATE TABLE post_views_2024_q4
    PARTITION OF post_views
    FOR VALUES FROM ('2024-10-01') TO ('2025-01-01');

CREATE TABLE post_views_2025_q1
    PARTITION OF post_views
    FOR VALUES FROM ('2025-01-01') TO ('2025-04-01');

CREATE INDEX idx_views_post ON post_views (post_id, viewed_at DESC);

-- -----
-- COMMENTS
-- -----

CREATE TABLE comments (
    comment_id    BIGSERIAL PRIMARY KEY,
    post_id       INTEGER NOT NULL REFERENCES posts(post_id) ON DELETE CASCADE,
    author_id     INTEGER NOT NULL REFERENCES users(user_id),
    parent_id     BIGINT REFERENCES comments(comment_id), -- for threading
    content       TEXT NOT NULL,
    is_deleted    BOOLEAN NOT NULL DEFAULT FALSE,
    reaction_count INTEGER NOT NULL DEFAULT 0,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_comments_post  ON comments (post_id, created_at);
CREATE INDEX idx_comments_parent ON comments (parent_id);
CREATE INDEX idx_comments_author ON comments (author_id);

-- -----
-- REACTIONS (polymorphic)

```

```

-----
CREATE TABLE reactions (
  reaction_id    BIGSERIAL PRIMARY KEY,
  user_id        INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  target_type    VARCHAR(20) NOT NULL CHECK (target_type IN ('post','comment')),
  target_id      BIGINT NOT NULL,
  reaction_type  VARCHAR(20) NOT NULL DEFAULT 'like'
                  CHECK (reaction_type IN ('like','love','fire','unicorn','clap')),
  created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (user_id, target_type, target_id)
);

CREATE INDEX idx_reactions_target ON reactions (target_type, target_id);

-----
-- BOOKMARKS
-----

CREATE TABLE bookmarks (
  user_id    INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  post_id    INTEGER NOT NULL REFERENCES posts(post_id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (user_id, post_id)
);

-----
-- NOTIFICATIONS
-----

CREATE TABLE notifications (
  notification_id BIGSERIAL PRIMARY KEY,
  user_id         INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  actor_id        INTEGER REFERENCES users(user_id),
  type            VARCHAR(30) NOT NULL
                  CHECK (type IN ('new_follower','post_comment','post_reaction',
'comment_reaction','mention','new_post_from_following')),
  target_type     VARCHAR(20),
  target_id       BIGINT,
  is_read         BOOLEAN NOT NULL DEFAULT FALSE,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_notif_user  ON notifications (user_id, is_read, created_at DESC);
CREATE INDEX idx_notif_unread ON notifications (user_id) WHERE is_read = FALSE;

```

6. ASCII ER Diagram

```

USERS
+-----+
| user_id (PK)      |<---+
| username (UNIQUE) |   | follower_id / following_id

```

```

| email (UNIQUE) | | FOLLOWS
| follower_count | +-----+
| following_count | | follower_id(FK) |
+-----+-----+ | following_id(FK)|
| | +-----+
| +-----+
| | |
| v | v
POSTS SERIES
+-----+ +-----+
| post_id | | series_id|
| author_id| | author_id|
| slug | | title |
| title | +-----+
| content |
| status | TOPICS
| search_ | +-----+
| vector | | topic_id |
+-----+ +-----+ | slug |
| | +-----+
+-----+ |
| | v
v v TAGS
POST_TAGS POST_VIEWS +-----+
+-----+ +-----+ | tag_id |
|post_id | |post_id | | slug |
|tag_id | |viewer_id| | topic_id |
+-----+ |read_pct | +-----+
+-----+
POST_REVISIONS
+-----+
| revision_id (PK) |
| post_id (FK) |
| revision_num |
| content |
+-----+
COMMENTS (self-ref)
+-----+
| comment_id (PK) |<---+
| post_id (FK) | | parent_id (threading)
| author_id (FK) |-----+
| parent_id (FK) |
+-----+
|
v
REACTIONS
+-----+
| reaction_id (PK) |
| user_id (FK) |
| target_type | -- 'post' or 'comment'
| target_id |
| reaction_type |

```

+-----+	
NOTIFICATIONS	BOOKMARKS
+-----+	+-----+
notif_id(PK)	user_id(FK)
user_id(FK)	post_id(FK)
actor_id	+-----+
type	
is_read	
+-----+	

7. Sample Data Scripts

```
SET search_path = blog, public;

-- Users
INSERT INTO users (username, email, display_name, bio, twitter_handle) VALUES
('alice_dev', 'alice@blog.io', 'Alice Chen', 'Full-stack developer.
PostgreSQL enthusiast.', 'alicedev'),
('bob_writes', 'bob@blog.io', 'Bob Martinez', 'Data engineer. Write about
pipelines.', 'bobwrites'),
('carol_tech', 'carol@blog.io', 'Carol Lee', 'Open source contributor. Linux
nerd.', 'caroltech'),
('admin', 'admin@blog.io', 'Platform Admin', '', NULL);

-- Topics & Tags
INSERT INTO topics (slug, name, color_hex) VALUES
('databases', 'Databases', '#E74C3C'),
('programming', 'Programming', '#3498DB'),
('devops', 'DevOps', '#27AE60');

INSERT INTO tags (slug, name, topic_id) VALUES
('postgresql', 'PostgreSQL', 1), ('sql', 'SQL', 1), ('nosql', 'NoSQL', 1),
('python', 'Python', 2), ('javascript', 'JavaScript', 2),
('docker', 'Docker', 3), ('kubernetes', 'Kubernetes', 3);

-- Series
INSERT INTO series (author_id, slug, title) VALUES
(1, 'postgresql-deep-dive', 'PostgreSQL Deep Dive: From Basics to Internals');

-- Posts
INSERT INTO posts (author_id, slug, title, content, series_id, series_position,
status, published_at) VALUES
(1, 'postgresql-indexes-explained',
'PostgreSQL Indexes Explained',
'An index is a special data structure that helps PostgreSQL find rows faster
without scanning the entire table. In this article, we'll explore B-Tree, Hash,
GIN, and GiST indexes with real examples. Understanding when to use each type can
dramatically improve query performance...',
1, 1, 'published', NOW() - INTERVAL '7 days'),
```

```

(2, 'python-data-pipelines-with-postgres',
 'Building Data Pipelines with Python and PostgreSQL',
 'Data pipelines are the backbone of modern analytics. In this guide, we'll build
an end-to-end pipeline that extracts data from REST APIs, transforms it using
pandas, and loads it into PostgreSQL with proper error handling and retry logic...',
 NULL, NULL, 'published', NOW() - INTERVAL '3 days'),

(1, 'understanding-mvcc-in-postgresql',
 'Understanding MVCC in PostgreSQL',
 'Multi-Version Concurrency Control (MVCC) is one of PostgreSQL's most powerful
features. Instead of locking rows during reads, PostgreSQL keeps multiple versions
of each row, allowing readers and writers to never block each other...',
 1, 2, 'published', NOW() - INTERVAL '1 day'),

(3, 'docker-postgres-production',
 'Running PostgreSQL in Production with Docker',
 'Running databases in containers is controversial but increasingly common. This
post covers the pros and cons, how to configure persistence, memory limits, and WAL
archiving when running PostgreSQL in Docker or Kubernetes...',
 NULL, NULL, 'draft', NULL);

-- Update search vectors manually (normally done by trigger)
UPDATE posts SET search_vector = to_tsvector('english', title || ' ' ||
COALESCE(subtitle,'') || ' ' || content);

-- Post tags
INSERT INTO post_tags (post_id, tag_id) VALUES
(1,1),(1,2),(2,4),(2,1),(3,1),(3,2),(4,6),(4,1);

-- Follows
INSERT INTO follows (follower_id, following_id) VALUES
(2,1),(3,1),(4,1),(1,2),(3,2);

-- Comments
INSERT INTO comments (post_id, author_id, parent_id, content) VALUES
(1, 2, NULL, 'Excellent breakdown of GIN vs GiST. Finally understand when to use
each.'),
(1, 3, NULL, 'Would love to see a follow-up on BRIN indexes for time-series
data.'),
(1, 1, 1, 'Thanks! BRIN post is in my queue. Stay tuned.'),
(2, 1, NULL, 'Great pipeline architecture. Have you benchmarked COPY vs INSERT?'),
(3, 2, NULL, 'MVCC explanation is crystal clear. Bookmarking this.');
```

```

-- Reactions
INSERT INTO reactions (user_id, target_type, target_id, reaction_type) VALUES
(2, 'post', 1, 'fire'), (3, 'post', 1, 'clap'), (4, 'post', 1, 'like'),
(1, 'post', 2, 'like'), (3, 'post', 2, 'love'),
(2, 'post', 3, 'fire'), (4, 'post', 3, 'clap'),
(1, 'comment', 1, 'like'), (3, 'comment', 1, 'like');
```

```

-- Bookmarks
```

```

INSERT INTO bookmarks (user_id, post_id) VALUES (2,1),(3,1),(4,3),(1,2);

-- Post views
INSERT INTO post_views (post_id, viewer_id, read_pct, viewed_at) VALUES
  (1, 2, 100, NOW()-INTERVAL '6 days'), (1, 3, 80, NOW()-INTERVAL '5 days'),
  (1, 4, 95, NOW()-INTERVAL '4 days'), (1, NULL, 60, NOW()-INTERVAL '3 days'),
  (2, 1, 100, NOW()-INTERVAL '2 days'), (2, 3, 75, NOW()-INTERVAL '1 day'),
  (3, 2, 100, NOW()-INTERVAL '12 hours');

```

8. Core SQL Queries

```

SET search_path = blog, public;

-- -----
-- Q1: Published post feed with engagement stats
-- -----

SELECT
  p.slug,
  p.title,
  u.username AS author,
  u.display_name,
  p.published_at::DATE,
  p.estimated_read_min AS read_min,
  p.view_count,
  p.reaction_count,
  p.comment_count,
  ARRAY_AGG(t.slug ORDER BY t.slug) AS tags
FROM posts p
JOIN users u ON p.author_id = u.user_id
LEFT JOIN post_tags pt ON p.post_id = pt.post_id
LEFT JOIN tags t ON pt.tag_id = t.tag_id
WHERE p.status = 'published'
GROUP BY p.post_id, p.slug, p.title, u.username, u.display_name,
  p.published_at, p.estimated_read_min,
  p.view_count, p.reaction_count, p.comment_count
ORDER BY p.published_at DESC;

-- -----
-- Q2: Full-text search across posts
-- -----

SELECT
  p.title,
  u.username AS author,
  p.published_at::DATE,
  ts_rank(p.search_vector, query) AS rank,
  ts_headline('english', p.content, query,
    'MaxWords=40, MinWords=20, StartSel=<b>, StopSel=</b>') AS excerpt
FROM posts p
JOIN users u ON p.author_id = u.user_id,
  to_tsquery('english', 'postgresql & index') AS query

```

```
WHERE p.search_vector @@ query
      AND p.status = 'published'
ORDER BY rank DESC;
```

```
-----
-- Q3: Threaded comments for a post (recursive CTE)
-----
```

```
WITH RECURSIVE comment_tree AS (
    -- Root comments
    SELECT
        c.comment_id,
        c.parent_id,
        c.author_id,
        c.content,
        c.is_deleted,
        c.reaction_count,
        c.created_at,
        0 AS depth,
        ARRAY[c.comment_id] AS path
    FROM comments c
    WHERE c.post_id = 1 AND c.parent_id IS NULL

    UNION ALL

    SELECT
        c.comment_id,
        c.parent_id,
        c.author_id,
        c.content,
        c.is_deleted,
        c.reaction_count,
        c.created_at,
        ct.depth + 1,
        ct.path || c.comment_id
    FROM comments c
    JOIN comment_tree ct ON c.parent_id = ct.comment_id
)
SELECT
    ct.depth,
    REPEAT(' ', ct.depth) || u.username AS author,
    CASE WHEN ct.is_deleted THEN '[deleted]' ELSE ct.content END AS content,
    ct.reaction_count,
    ct.created_at::DATE
FROM comment_tree ct
JOIN users u ON ct.author_id = u.user_id
ORDER BY ct.path;
```

```
-----
-- Q4: Personalized feed (posts from followed authors)
-----
```

```
SELECT
    p.title,
```

```

    u.username          AS author,
    u.display_name,
    p.published_at,
    p.estimated_read_min,
    p.view_count,
    p.reaction_count
FROM follows f
JOIN posts p  ON f.following_id = p.author_id
JOIN users u  ON p.author_id    = u.user_id
WHERE f.follower_id = 1
      AND p.status = 'published'
      AND p.published_at >= NOW() - INTERVAL '30 days'
ORDER BY p.published_at DESC
LIMIT 20;

-----
-- Q5: Trending posts (engagement score last 7 days)
-----

WITH recent_metrics AS (
    SELECT
        p.post_id,
        COUNT(DISTINCT pv.view_id)           AS recent_views,
        COUNT(DISTINCT r.reaction_id)        AS recent_reactions,
        COUNT(DISTINCT c.comment_id)         AS recent_comments
    FROM posts p
    LEFT JOIN post_views pv ON p.post_id = pv.post_id
                        AND pv.viewed_at >= NOW() - INTERVAL '7 days'
    LEFT JOIN reactions r  ON r.target_type = 'post'
                        AND r.target_id = p.post_id
                        AND r.created_at >= NOW() - INTERVAL '7 days'
    LEFT JOIN comments c   ON p.post_id = c.post_id
                        AND c.created_at >= NOW() - INTERVAL '7 days'
    WHERE p.status = 'published'
    GROUP BY p.post_id
)
SELECT
    p.title,
    u.username          AS author,
    rm.recent_views,
    rm.recent_reactions,
    rm.recent_comments,
    -- Weighted engagement score
    rm.recent_views + (rm.recent_reactions * 3) + (rm.recent_comments * 5) AS
engagement_score
FROM recent_metrics rm
JOIN posts p  ON rm.post_id = p.post_id
JOIN users u  ON p.author_id = u.user_id
ORDER BY engagement_score DESC
LIMIT 10;

-----
-- Q6: Author analytics dashboard

```

```

-----
SELECT
    p.title,
    p.published_at::DATE,
    p.estimated_read_min,
    COUNT(DISTINCT pv.view_id)           AS total_views,
    ROUND(AVG(pv.read_pct), 1)          AS avg_read_pct,
    p.reaction_count,
    p.comment_count,
    COUNT(DISTINCT b.user_id)           AS bookmarks
FROM posts p
LEFT JOIN post_views pv ON p.post_id = pv.post_id
LEFT JOIN bookmarks b   ON p.post_id = b.post_id
WHERE p.author_id = 1
      AND p.status = 'published'
GROUP BY p.post_id, p.title, p.published_at, p.estimated_read_min,
         p.reaction_count, p.comment_count
ORDER BY total_views DESC;

```

```

-----
-- Q7: Tag popularity and post count
-----

```

```

SELECT
    t.slug,
    t.name,
    top.name           AS topic,
    COUNT(pt.post_id)  AS post_count,
    SUM(p.view_count)  AS total_views,
    RANK() OVER (ORDER BY COUNT(pt.post_id) DESC) AS rank
FROM tags t
LEFT JOIN topics top ON t.topic_id = top.topic_id
LEFT JOIN post_tags pt ON t.tag_id = pt.tag_id
LEFT JOIN posts p      ON pt.post_id = p.post_id AND p.status = 'published'
GROUP BY t.tag_id, t.slug, t.name, top.name
ORDER BY post_count DESC;

```

```

-----
-- Q8: User profile stats
-----

```

```

SELECT
    u.username,
    u.display_name,
    u.follower_count,
    u.following_count,
    COUNT(DISTINCT p.post_id) FILTER (WHERE p.status = 'published') AS
published_posts,
    SUM(p.view_count)           AS total_views,
    SUM(p.reaction_count)       AS
total_reactions,
    MIN(p.published_at)::DATE    AS first_post,
    MAX(p.published_at)::DATE    AS latest_post
FROM users u

```

```

LEFT JOIN posts p ON u.user_id = p.author_id
WHERE u.user_id = 1
GROUP BY u.user_id, u.username, u.display_name, u.follower_count, u.following_count;

```

```

-----
-- Q9: Posts with revision history
-----

```

```

SELECT
    p.title,
    pr.revision_num,
    pr.revised_at::DATE,
    u.username AS revised_by,
    pr.change_summary,
    LENGTH(pr.content) - COALESCE(
        LAG(LENGTH(pr.content)) OVER (PARTITION BY pr.post_id ORDER BY
pr.revision_num), 0
    ) AS chars_changed
FROM post_revisions pr
JOIN posts p ON pr.post_id = p.post_id
JOIN users u ON pr.revised_by = u.user_id
WHERE pr.post_id = 1
ORDER BY pr.revision_num DESC;

```

```

-----
-- Q10: Series reading progress for a user
-----

```

```

SELECT
    s.title AS series,
    p.series_position AS part,
    p.title AS post_title,
    p.estimated_read_min,
    CASE WHEN pv.view_id IS NOT NULL THEN 'Read' ELSE 'Unread' END AS status,
    pv.read_pct
FROM series s
JOIN posts p ON s.series_id = p.series_id
LEFT JOIN post_views pv ON p.post_id = pv.post_id AND pv.viewer_id = 2
WHERE s.series_id = 1
ORDER BY p.series_position;

```

```

-----
-- Q11: Unread notifications for a user
-----

```

```

SELECT
    n.type,
    u_actor.username AS from_user,
    n.target_type,
    n.target_id,
    n.created_at
FROM notifications n
LEFT JOIN users u_actor ON n.actor_id = u_actor.user_id
WHERE n.user_id = 1 AND n.is_read = FALSE
ORDER BY n.created_at DESC;

```

```

-- -----
-- Q12: View velocity - posts gaining momentum
-- -----

SELECT
    p.title,
    COUNT(pv.view_id) FILTER (WHERE pv.viewed_at >= NOW() - INTERVAL '24 hours')
AS views_24h,
    COUNT(pv.view_id) FILTER (WHERE pv.viewed_at >= NOW() - INTERVAL '48 hours'
                                AND pv.viewed_at < NOW() - INTERVAL '24 hours')
AS views_prev_24h,
    ROUND(
        100.0 * (
            COUNT(pv.view_id) FILTER (WHERE pv.viewed_at >= NOW() - INTERVAL '24
hours')
            - COUNT(pv.view_id) FILTER (WHERE pv.viewed_at >= NOW() - INTERVAL '48
hours'
                                AND pv.viewed_at < NOW() - INTERVAL '24
hours')
        )
        / NULLIF(COUNT(pv.view_id) FILTER (WHERE pv.viewed_at >= NOW() - INTERVAL
'48 hours'
                                AND pv.viewed_at < NOW() - INTERVAL '24
hours'), 0)
        , 1) AS growth_pct
FROM posts p
LEFT JOIN post_views pv ON p.post_id = pv.post_id
WHERE p.status = 'published'
GROUP BY p.post_id, p.title
ORDER BY views_24h DESC;

```

9. Stored Procedures / Functions

```

-- -----
-- FUNCTION 1: Publish a post (validate + set timestamps)
-- -----

CREATE OR REPLACE FUNCTION blog.publish_post(
    p_post_id INTEGER,
    p_author_id INTEGER
)
RETURNS TEXT
LANGUAGE plpgsql AS $$
DECLARE
    v_post blog.posts%ROWTYPE;
    v_rev_num SMALLINT;
BEGIN
    SELECT * INTO v_post FROM blog.posts WHERE post_id = p_post_id FOR UPDATE;
    IF NOT FOUND THEN RAISE EXCEPTION 'Post % not found', p_post_id; END IF;
    IF v_post.author_id <> p_author_id THEN
        RAISE EXCEPTION 'User % is not the author of post %', p_author_id,

```

```

p_post_id;
END IF;
IF v_post.status = 'published' THEN
    RAISE EXCEPTION 'Post % is already published', p_post_id;
END IF;
IF LENGTH(TRIM(v_post.title)) < 5 THEN
    RAISE EXCEPTION 'Post title is too short';
END IF;
IF LENGTH(TRIM(v_post.content)) < 100 THEN
    RAISE EXCEPTION 'Post content is too short (minimum 100 characters)';
END IF;

-- Save revision before publishing
SELECT COALESCE(MAX(revision_num), 0) + 1 INTO v_rev_num
FROM blog.post_revisions WHERE post_id = p_post_id;

INSERT INTO blog.post_revisions (post_id, revision_num, title, content,
revised_by, change_summary)
VALUES (p_post_id, v_rev_num, v_post.title, v_post.content, p_author_id,
'Published');

UPDATE blog.posts
SET status = 'published',
    published_at = NOW(),
    updated_at = NOW(),
    search_vector = to_tsvector('english',
        v_post.title || ' ' || COALESCE(v_post.subtitle, '') || ' ' ||
v_post.content)
WHERE post_id = p_post_id;

-- Update user post count
UPDATE blog.users SET post_count = post_count + 1 WHERE user_id = p_author_id;

RETURN format('Post "%s" published successfully', v_post.title);
END;
$$;

-- -----
-- FUNCTION 2: Fan-out notifications on new post
-- -----

CREATE OR REPLACE FUNCTION blog.notify_followers_new_post(p_post_id INTEGER)
RETURNS INTEGER
LANGUAGE plpgsql AS $$
DECLARE
    v_post      blog.posts%ROWTYPE;
    v_notif_count INTEGER := 0;
BEGIN
    SELECT * INTO v_post FROM blog.posts WHERE post_id = p_post_id;
    IF NOT FOUND OR v_post.status <> 'published' THEN RETURN 0; END IF;

    INSERT INTO blog.notifications (user_id, actor_id, type, target_type, target_id)
    SELECT f.follower_id, v_post.author_id, 'new_post_from_following', 'post',

```

```

p_post_id
FROM blog.follows f
WHERE f.following_id = v_post.author_id
AND NOT EXISTS (
    SELECT 1 FROM blog.notifications n
    WHERE n.user_id = f.follower_id
    AND n.target_type = 'post'
    AND n.target_id = p_post_id
    AND n.type = 'new_post_from_following'
);

GET DIAGNOSTICS v_notif_count = ROW_COUNT;
RETURN v_notif_count;
END;
$$;

-- -----
-- FUNCTION 3: Record a post view
-- -----

CREATE OR REPLACE FUNCTION blog.record_view(
    p_post_id    INTEGER,
    p_viewer_id  INTEGER DEFAULT NULL,
    p_session_hash VARCHAR DEFAULT NULL,
    p_read_pct   SMALLINT DEFAULT 0
)
RETURNS VOID
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO blog.post_views (post_id, viewer_id, session_hash, read_pct)
    VALUES (p_post_id, p_viewer_id, p_session_hash, p_read_pct);

    UPDATE blog.posts
    SET view_count = view_count + 1
    WHERE post_id = p_post_id;
END;
$$;

```

10. Triggers

```

-- TRIGGER 1: Auto-update search_vector on post save
CREATE OR REPLACE FUNCTION blog.trg_update_search_vector()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    NEW.search_vector := to_tsvector('english',
        NEW.title || ' '
        || COALESCE(NEW.subtitle, '') || ' '
        || NEW.content
    );
    NEW.updated_at := NOW();
    RETURN NEW;

```

```

END;
$$;

CREATE TRIGGER trg_posts_search_vector
    BEFORE INSERT OR UPDATE OF title, subtitle, content ON blog.posts
    FOR EACH ROW EXECUTE FUNCTION blog.trg_update_search_vector();

-- TRIGGER 2: Update reaction_count on post after reaction insert/delete
CREATE OR REPLACE FUNCTION blog.trg_sync_reaction_count()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'INSERT' AND NEW.target_type = 'post' THEN
        UPDATE blog.posts SET reaction_count = reaction_count + 1 WHERE post_id =
NEW.target_id;
        -- Notify post author
        INSERT INTO blog.notifications (user_id, actor_id, type, target_type,
target_id)
        SELECT p.author_id, NEW.user_id, 'post_reaction', 'post', NEW.target_id
        FROM blog.posts p
        WHERE p.post_id = NEW.target_id AND p.author_id <> NEW.user_id;
    ELSIF TG_OP = 'DELETE' AND OLD.target_type = 'post' THEN
        UPDATE blog.posts SET reaction_count = GREATEST(0, reaction_count - 1) WHERE
post_id = OLD.target_id;
    END IF;
    RETURN COALESCE(NEW, OLD);
END;
$$;

CREATE TRIGGER trg_reactions_post_count
    AFTER INSERT OR DELETE ON blog.reactions
    FOR EACH ROW EXECUTE FUNCTION blog.trg_sync_reaction_count();

```

11. Performance Optimization

```

-- Materialized view for trending posts (refresh every hour)
CREATE MATERIALIZED VIEW blog.mv_trending_posts AS
SELECT p.post_id, p.title, p.author_id,
    COUNT(pv.view_id) AS views_7d,
    p.reaction_count,
    p.comment_count,
    COUNT(pv.view_id) + (p.reaction_count * 3) + (p.comment_count * 5) AS score
FROM blog.posts p
LEFT JOIN blog.post_views pv ON p.post_id = pv.post_id
    AND pv.viewed_at >= NOW() - INTERVAL '7 days'
WHERE p.status = 'published'
GROUP BY p.post_id, p.title, p.author_id, p.reaction_count, p.comment_count
WITH DATA;

```

```
CREATE UNIQUE INDEX ON blog.mv_trending_posts (post_id);
CREATE INDEX ON blog.mv_trending_posts (score DESC);
```

Strategy	Details
GIN on search_vector	Sub-millisecond full-text search
Partitioned post_views	Keeps view table manageable by quarter
Materialized view	Trending score never computed at query time
Partial index on status='published'	Feed queries only touch published posts
Denormalized counters	view_count, reaction_count, comment_count on posts

12. Extensions Used

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;           -- fuzzy author/tag search
CREATE EXTENSION IF NOT EXISTS unaccent;          -- normalize accent characters in
FTS
CREATE EXTENSION IF NOT EXISTS pg_cron;           -- schedule materialized view
refresh

-- Use unaccent in FTS configuration
CREATE TEXT SEARCH CONFIGURATION blog_fts (COPY = english);
ALTER TEXT SEARCH CONFIGURATION blog_fts
    ALTER MAPPING FOR hword, hword_part, word WITH unaccent, english_stem;

SELECT cron.schedule('refresh-trending', '0 * * * *',
    'REFRESH MATERIALIZED VIEW CONCURRENTLY blog.mv_trending_posts');
```

13. Testing Guide

```
-- TEST 1: Publish a post
SELECT blog.publish_post(4, 3); -- carol's draft post

-- TEST 2: FTS search
SELECT title FROM blog.posts WHERE search_vector @@ to_tsquery('english',
'postgresql & index');

-- TEST 3: Threaded comments (recursive CTE)
-- Insert a 3-level deep comment thread and run Q3

-- TEST 4: Personalized feed
-- Follow user 1 as user 2 and verify feed shows posts

-- TEST 5: Reaction count trigger
INSERT INTO blog.reactions (user_id, target_type, target_id, reaction_type) VALUES
(4, 'post', 2, 'fire');
```

```

SELECT reaction_count FROM blog.posts WHERE post_id = 2; -- Should increment

-- TEST 6: Notification fan-out
SELECT blog.notify_followers_new_post(1);
SELECT * FROM blog.notifications WHERE user_id = 2 ORDER BY created_at DESC LIMIT 1;

```

14. Extension Challenges

- Monetization / Tipping:** Add a `tips` table allowing readers to send micropayments to authors. Calculate author earnings, payout thresholds, and monthly earning reports.
- Sponsored Content:** Add a `sponsors` table and `sponsored_posts` linking. Track impressions vs. paid views. Write a billing report for sponsors based on actual delivery.
- Reading Lists / Collections:** Let users create public or private reading lists beyond bookmarks. Add many-to-many posts-to-lists. Write a "list recommendation" function based on content overlap.
- Moderation Queue:** Add a `moderation_queue` for flagged comments and posts. Build a triage workflow with priority scoring. Track moderator decisions and appeal history.
- Multilingual FTS:** Support posts in multiple languages. Detect language using a lookup table and configure the `search_vector` to use the appropriate text search configuration per language.

15. What You Learned

Skill	Demonstrated By
Full-text search with GIN	<code>search_vector</code> column + <code>to_tsquery</code>
Threaded comments	Recursive CTE with adjacency list
Table partitioning	<code>post_views</code> partitioned by quarter
Generated columns	<code>estimated_read_min</code> computed from content length
Polymorphic relationships	<code>reactions</code> and <code>notifications</code> <code>target_type/id</code> pattern
Materialized views	Trending posts score cached and refreshed
Denormalized counters	<code>view_count</code> , <code>reaction_count</code> updated via triggers
Fan-out notifications	Batch INSERT from followers list
Array operations	Tags as <code>ARRAY_AGG</code> in feed query
Text search configuration	Custom FTS with <code>unaccent</code>